

Impact of user skills and network effects on the competition between open source and proprietary software

Lihui Lin

Boston University School of Management, Boston, MA 02215, United States

Received 15 January 2007; accepted 16 January 2007

Available online 25 February 2007

Abstract

Despite the availability of source code, deploying open source software in an enterprise environment requires skills. Recent surveys and case studies show that the skills of IT staff in an organization are one of the key factors in open source software adoption decisions. Another important factor organizations consider when choosing a platform is network effects. This paper studies how users' skills and network effects may influence the market where proprietary software competes with open source software. In the model, users make adoption decisions considering their own skills and the network effects of the software, and the proprietary software vendor prices its product strategically. It is found that in the presence of network effects, the proprietary software dominates the market when the open source software does not provide higher benefits to users. This implies that open source software, as a low-cost substitute to proprietary software, may not survive in a market exhibiting network effects. To gain market share, open source software has to outperform proprietary software by a large margin, which can be achieved when a significant portion of users are highly skilled and thus can customize the open source software to better satisfy their needs.

© 2007 Elsevier B.V. All rights reserved.

Keywords: Duopoly; Economic analysis; Market structure; Network effect; Open source software; Open source software skills; User customization; User heterogeneity

1. Introduction

Open source software is one of the most discussed subjects in recent years. What attracted researchers' attention initially was the unique open source approach of software production. Early research focuses on issues such as what motivates open source contributors and what the implications are of open source production. Research has attempted to explain the motivations for participating in open source software projects from the angles of labor economics, public good economics, and industrial organization [16,22,25,26,39,43].

With its rapid proliferation, open source software is now competing with proprietary software in many segments of software market. According to Netcraft's surveys, more than 60% of websites employ the open source web server

software Apache, while about 30% use Microsoft's Internet Information Services (IIS) [34]. The open source GNU/Linux operating system (hereafter referred to as Linux) holds approximately a 30% share in the market of server operating systems, where Microsoft's Windows holds about a 50% share [33]. While the advantages of both types of software are discussed in the literature, direct competition between the two paradigms has received relatively little attention [26]. In the limited prior research about this issue, the two types of software are treated as products with different features, reflecting the focus on the supply side in research about open source [18,38].

This paper studies the competition between open source and proprietary software with an emphasis on the *demand* side. Specifically, I focus on the impact of users' open source software skills on firms' software adoption decisions. The important role of users' skills in these decisions is confirmed by the following empirical facts.

E-mail address: lhlin@bu.edu

First, early adopters of open source software have primarily been developers and expert users; however, this profile has changed drastically as open source software has become mainstream in software markets [14]. Linux, for example, is now used by organizations of all sizes in various industries. Data from Business Source Premier show that organizations which adopted Linux between 2000 and 2004 include carmakers, oil companies, manufacturers of home and personal care products, pharmaceutical firms, supermarket chains, travel agencies, investment banks, insurance companies, movie studios, libraries, schools, and government agencies. Kusnetzky and Gillen [24] report similar findings. The diversity of firms using open source software calls for investigation of the key factors in their adoption decisions.

Second, most firms considering adopting open source software are primarily concerned with the implementation and support cost for this software. Linux is considered more difficult to set up and install than Windows and supporting Linux in an enterprise environment requires special skills. In a recent survey of nearly 20,000 enterprises, 48% of mid-sized enterprises show no interest in Linux, and a major impediment is the cost of support for Linux [21]. The survey also shows that of the mid-sized enterprises that have implemented Linux, there is a skew towards the high-end. This is because to adopt Linux, smaller organizations often have to add new or retrain existing information technology (IT) personnel, significantly increasing IT cost, while larger organizations with more IT staff are less sensitive to such an increase in personnel cost and also more likely to have in-house expertise for Linux. These survey results are consistent with the total cost of ownership (TCO) analysis of software adoption [12], which emphasizes that acquisition cost is only a small portion of the total cost of owning software, and thus that organizations should consider the TCO, predominantly consisting of ongoing operational costs such as maintenance and support. Several recent studies compare the TCO of Linux and proprietary operating systems [11,19,29,30,36,42].

Third, empirical evidence also shows that the total cost of deploying open source software differs across firms and depends largely on the skills and expertise of the IT staff in an organization. Intuitively, if the IT staff of a firm are active contributors to an open source project, the support cost for this software tends to be very low. Greenemeier and Foley [20] provide business cases where companies fail to consider the skills required for open source software deployments and subsequently incur costs much higher than expected. It must be noted, though, that firms do not have to support open source software internally; instead, they can acquire external support from third parties through service contracts. Reports show that a Linux system with an external support contract costs more than one supported internally [36]. This suggests that inadequate in-house skills may cause a firm to resort to external support, which likely results in higher costs. In addition, hav-

ing in-house skills may lower the total cost even for firms using third-party support. The leading Linux provider Red Hat, for example, offers a pricing scheme that increases with its service level, so firms with more skilled personnel can save costs by choosing a lower level of service from Red Hat and using internal staff to provide any outstanding support needed.¹

Summing up the above empirical evidence, firms are heterogeneous in their open source software skills, and a firm's level of skills is one of the key factors in determining the cost of deploying open source software. One of the key reasons is that support for open source software does not have to be provided by a vendor: adopting firms can deploy the system themselves. As a result, the major concern in open source software adoption decision is *support* rather than *acquisition*. Since the cost of open source software mainly comes from maintenance and support, provided either in-house or externally, the level of skills becomes a distinguishing factor for open source software deployment. On the contrary, when firms deploy proprietary software, the cost lies mostly in acquisition. Even though firms may gain different levels of skill with the proprietary software, all firms rely on the vendor for new releases and upgrades. Therefore, the impact of skills on the total cost of ownership is much less for proprietary software than for open source software.

Taking into account these empirical facts, I model users in the software market as heterogeneous in their levels of open source skills, which determine their cost of deploying the open source software. Users' heterogeneity in open source software deployment cost is one of the key features studied in this paper. Previous research mostly abstracts from different skill levels of open source software users and assumes the same open source software implementation costs for all users [1,13,32].² In this paper, users face the same cost only if they choose proprietary software, where the cost is fully represented by the price charged by the vendor.³

Users' skills are also fundamental to the benefits provided by open source software for its adopters. Open source advocates claim that firms using proprietary software lose control of their business to the vendor, while open source software, where the source code remains

¹ See <http://www.redhat.com> for Red Hat's price structure. One may argue that the choice of service level depends on the purpose of a system rather than the skills of an adopter. What this paper is interested in, however, is that for firms using a system for the same purpose, high level of skills does allow a firm to save costs. For example, Red Hat's pricing schedule shows that the basic service provides only 30 days of telephone support while higher levels of service include 1-year telephone support. For the same use of a system, a firm with more skilled staff can choose the basic service because the skilled staff tends to learn much faster.

² In Bessen's model [1], users are heterogeneous in requirements or "tastes" for software but homogeneous in their ability to develop open source software.

³ Note that the price for the proprietary software is *not* just for the software itself – it also includes warranties and service fees charged by the vendor.

open, provides its users with the ultimate freedom [35]. Users may modify standard software to meet their specific needs. This process is called *customization* [17]. With its source code available, open source software, in theory, allows any feasible customization. In reality, however, many organizations lack the competence or even the desire to inspect or modify the source code, a phenomenon called the *Berkeley Conundrum* [15]. This puzzle cannot be solved without considering users' knowledge and skills. Empirical research has found that only highly skilled open source software users customize the software. A case study of Apache security software finds that the degree of software customization by users has strong positive correlation with users' skills, and that skilled users who customize the software are more satisfied with it [17]. This suggests that skilled users can reap higher value by customizing open source software. The study also shows, however, that some skilled users choose not to customize when the costs of customization (in terms of working hours of skilled programmers) exceed the benefits [17]. Here I assume that the benefits users derive from open source software depend on the degree of customization of the open source software, and the cost of customization depends on both the degree of customization and users' skills.⁴

Another important factor in users' software adoption decisions is network effects. Network effects arise both directly from compatibility with other users of the same software and indirectly from availability of complementary goods [23,27]. The literature has long recognized the importance of network effects in the software market [4,6,40,41]. Operating systems such as Windows and Linux exhibit strong network effects. An operating system is more valuable to each user when more users adopt it, because it is more likely for software and hardware vendors to develop applications and devices (complementary goods) for this system. Application software also has network value, because its users benefit from the ability to communicate and collaborate with other users of the same software. In fact, a study of software adoption without considering network effects would be incomplete. In particular, Bonaccorsi and Rossi [2] call for research on the role of network effects in the diffusion of open source software in an environment dominated by established proprietary standards.

This paper investigates the competition between proprietary and open source software where users with different levels of skills compare the two types of software, taking into account the costs and benefits of customizing open source software and the network effects of software. The keys results are as follows.

First, in the absence of network effects, open source software and proprietary software will share the market. Furthermore, open source software gains market share as users become more skilled in it. Second, when the two types of software provide not only standalone benefits but also network value to users, the proprietary software vendor may drive open source software out of the market and become the monopoly. This is because the proprietary software vendor can price its product strategically to retain or capture market share. More surprisingly, *the proprietary software vendor takes over the entire market when users are highly skilled in open source software*. The reason is as follows: when users are less skilled in open source software, open source software is less attractive for potential adopters, and therefore, the vendor may charge a high price for the proprietary software to maximize profit, allowing the open source software to cover a portion of the market. However, when many IT professionals have gained open source software skills, open source software becomes a feasible and attractive choice for many users, forcing the vendor of proprietary software to reduce price. At the reduced price, it is more profitable for the proprietary software vendor to cover the entire market than to limit the supply of the software. This result suggests that in the presence of network effects, while open source software may gain a significant market share in the beginning of the competition, its existence and threat will force proprietary software providers to lower prices, leading the market to tip in favor of the proprietary software again.

The last set of key results shows that open source software may dominate the market only if it provides benefits beyond what its proprietary counterpart can provide. It is not surprising that in the absence of network effects, the market may favor open source software that provides higher value to users. With network effects, however, proprietary software may cover the entire market even when customized open source software provides much better performance to skilled users. The open source software gains market share only when there are enough skilled users who find customizing the open source software worthwhile.

This research sheds light on the diffusion of open source software in an environment predominated by proprietary software. In related research, Casadesus-Masanell and Ghemawat [7] develop a dynamic mixed duopoly model and show that open source software and proprietary software coexist even when open source software benefits users more. Their paper posits that superior demand-side learning in open source software development leads to higher quality software, while in this paper, the higher benefits of open source software to users stem from customization, which ultimately depends on users' skills. One implication of this paper is that the key to the successful dissemination of open source software is the customizability of the software, and more specifically, the cost and benefits of customization to users of different skill levels.

⁴ The benefits of open source software to users may also depend on users' specific requirements (e.g., firms in the financial industry often require high security). However, this paper focuses on the effect of users' expertise, therefore users are considered as heterogeneous only in terms of expertise. The users can be considered as having similar requirements (e.g., in the same industry).

2. Competition between proprietary and open source software without customization

2.1. Basic model setup

Each user has an inelastic unit demand for a piece of software. Users can be interpreted as individuals or enterprises with needs for some software (e.g., an operating system, an application, or a tailored program/project). There are two types of software in the market that can satisfy a user's need: S_1 and S_2 .

S_1 is proprietary software provided by a profit-maximizing firm referred to as the “proprietary software vendor” or simply “vendor”. Because the vendor controls the property rights to S_1 , it is the sponsor of this software [23]. S_1 provides its user a standalone value of s_1 . (See Table 1 for a complete list of notations.) Standalone value excludes any network value. The vendor charges a price of p to all users, assuming that it cannot price discriminate. The marginal cost of providing the proprietary software is assumed to equal zero.

S_2 is open source software for which the source code is freely available. S_2 is an unsponsored technology because there is no entry barrier to its supply [23]. In fact, S_2 does not even need to be supplied by a vendor. When deployed, S_2 provides its user a standalone value of s_2 . However, it requires skills to deploy.⁵

The model assumes that each user is endowed with a level of skills for the open source software, θ , also referred to as the type of a user. Users are uniformly distributed and the total mass of users is normalized to 1. This means $\theta \sim U[0, 1]$.⁶ Each user knows the realization of its type. A user's cost of deploying S_2 depends on his level of skills: the higher the skill level, the lower the cost. The cost of deploying S_2 can be denoted by $c(\theta)$, and $c'(\theta) < 0$.

In this section, deploying either one of the systems S_1 and S_2 provides the same standalone value, that is, $s_1 = s_2 = s$. It can be understood as: for a user of type θ , for any proprietary software he may choose to adopt, there is open source software he can deploy at a cost of $c(\theta)$ to obtain the same standalone benefits.

The sequence of events goes as follows. First, the proprietary software vendor chooses the profit-maximizing price. Then all users make their adoption decisions simultaneously. A user chooses the system that maximizes his utility. The user gets a reservation utility of zero when he stays out of the market (chooses neither of the systems). When the maximum utility a user derives from any system is zero, I assume the user chooses to deploy the system rather than staying out of the market.

2.2. Base case

In the simplest case, the net benefits a type θ user gains from deploying S_1 and S_2 are given by $u_1 = s - p$ and $u_2 = s - c(\theta)$.

We now discuss the meaning and properties of $c(\theta)$. The cost of implementing S_2 , $c(\theta)$, can be in the form of learning cost, personnel cost for hiring skilled staff, or outsourcing cost if the implementation task is outsourced. The cost is ultimately determined by the learning cost.⁷ The definition of θ requires that $c'(\theta) < 0$, that is, open source software implementation costs decrease with a user's skill level. The next question is, how does the marginal cost change with user's skills? Early economics literature on organizational learning assumes that a firm's cumulative knowledge, similar to other input factors, exhibits diminishing marginal returns [37]. This calls for a convex cost function, $c''(\theta) > 0$, which means that as users become more skilled, the cost savings from additional skills become less significant. In contrast, the psychology literature suggests that prior knowledge enhances learning [3]. In their seminal work, Cohen and Levinthal [9] introduce the notion of absorptive capacity to the management literature (for a recent survey, see [45]). The absorptive capacity theory of individual and organizational learning suggests that prior knowledge enhances the ability of a person or organization to assimilate and exploit new knowledge. This implies that for a skilled user, a unit gain in knowledge can save more cost than for a less skilled user, which translates mathematically to $c''(\theta) < 0$.

The following functional form captures the properties discussed above:

$$c(\theta) = (1 - \theta^\alpha)C, \quad \text{where } C > 0, \alpha > 0$$

First, this function satisfies the condition $c'(\theta) < 0$. This cost function also normalizes the cost incurred by a user with the highest skill level to zero, $c(\theta = 1) = 0$.⁸ Furthermore, the constant C represents the cost incurred by a user with no skills ($\theta = 0$), which is the maximum cost of implementing open source software.

Second, a nice feature of this specification is that the value of the parameter α reflects underlying assumptions about the shape of the cost function. When $0 < \alpha < 1$, $c'(\theta)|_{\theta \rightarrow 0+} \rightarrow -\infty$ and $c''(\theta) > 0$, which means the cost function exhibits diminishing marginal returns of skills. For a user with no skills, the initial knowledge and skills acquired save tremendous cost in system support, while the cost saving is less dramatic for more skilled users.

⁵ An individual user's skill is easy to understand. For an organizational user, the level of skills depends on the collective knowledge and skills of the staff in charge of the software.

⁶ Other distributions make the analysis more complicated without providing significantly new insights.

⁷ When a user hires employees with skills in S_2 , the employees need to be compensated for their learning cost. When a user resorts to external support, the provider demands compensation that reflects its skills, which was acquired through learning in the past.

⁸ Users with the highest level of expertise can be regarded as firms that have hired (or individuals who are) active and contributing members to the open source software being adopted, and for these users implementing and maintaining the software does not cost more than installing and maintaining proprietary software.

Table 1
Notation

s_1	The standalone value of the proprietary software S_1
p	The price of the proprietary software S_1
p^*	The equilibrium price of the proprietary software S_1
p^M	The price of the proprietary software S_1 if the proprietary software vendor is the monopolist
s_2	The standalone value of the open source software S_2
s	The standalone value of the software when the two types of software are perfect substitutes, $s_1 = s_2 = s$
θ	A user's level of skill for the open source software S_2 , also referred to as the type of a user. θ is distributed uniformly in the interval $[0, 1]$
$c(\theta)$	A user's cost of deploying S_2 , which is a function of its level of skill (or type) θ . The functional form used is: $c(\theta) = (1 - \theta^\alpha)C$
C	A parameter that represents the cost for a user with no open source skills ($\theta = 0$) to implement the open source software, that is, $c(\theta = 0) = C$
α	A parameter that reflects the rate of returns (reflected in the cost savings) of open source skills
u_1	The net benefits a user derives from the proprietary software
u_2	The net benefits a user derives from the open source software
$\tilde{\theta}$	The type of the user who is indifferent between the proprietary and the open source software. Because of the uniform distribution assumption of θ , $\tilde{\theta}$ also represents the market share of the proprietary software
$\tilde{\theta}^*$	The equilibrium market share of the proprietary software
π	Profit of the proprietary software vendor
π^*	The equilibrium profit of the proprietary software vendor
B	The intensity of network effects
γ	A parameter that reflects the diminishing marginal network value of software
d	The degree of customization for the open source software, $d \in [0, 1]$
d^*	A user's optimal customization level
$r(d)$	The benefits a user derives from the customization of open source software. The functional form used is: $r(d) = d \cdot R$
R	The highest benefits a user can derive from customizing open source software
$c(\theta, d)$	The cost of deploying open source software for a type θ user with a customization level of d . The functional form used is: $c(\theta, d) = (1 - \theta^\alpha)(C + d \cdot D)$
D	The maximum cost of customization incurred by a user with no open source software skills
$\hat{\theta}$	The type of the user who is indifferent between full customization ($d = 1$) and no customization ($d = 0$) of the open source software
ρ	The ratio between the benefits and costs of full customization for a user with no open source software skills, that is, $\rho \equiv R/D$

When $\alpha = 1$, we have: $c'(\theta) = -C$ and $c''(\theta) = 0$. This implies that the cost savings for a unit gain in skill level are the same for users of all skill levels. When $\alpha > 1$, the cost function is consistent with the absorptive capacity theory [9]. Since $c'(\theta)|_{\theta \rightarrow 0+} = 0$ and $c''(\theta) < 0$, for a user with no skills, initial acquisition of skills does not reduce the cost immediately, while cost savings occur at a higher rate for more skilled users. In this paper, there are no restrictions on the range of α other than $\alpha \in (0, +\infty)$. The actual value of the parameter α would depend on the nature of the skills and the user population.

Using the above specification of $c(\theta)$, a user of type θ derives $u_2 = s - (1 - \theta^\alpha)C$ from S_2 . The user who is indifferent between the two systems is denoted by $\tilde{\theta}$, and

$$p = c(\tilde{\theta}) = (1 - \tilde{\theta}^\alpha)C$$

Obviously, any user with $\theta < \tilde{\theta}$ chooses S_1 and the rest choose S_2 . Therefore, the market share of S_1 is $\tilde{\theta}$ and that of S_2 is $1 - \tilde{\theta}$.

The proprietary software vendor sets p to maximize its profit. Note that p cannot exceed s . The vendor's problem, therefore, is given by:

$$\begin{aligned} \max_p \quad & \pi = \tilde{\theta} \cdot p \\ \text{s.t.} \quad & 0 \leq \tilde{\theta} \leq 1 \text{ and } 0 \leq p \leq s, \\ & \text{where } p = c(\tilde{\theta}) = (1 - \tilde{\theta}^\alpha)C \end{aligned}$$

Solving the above problem yields the equilibrium price and market share of the proprietary software:

$$\begin{aligned} p^* &= \frac{\alpha}{1 + \alpha}C, \quad \tilde{\theta}^* = (1 + \alpha)^{-1/\alpha} \text{ when } C \leq \left(1 + \frac{1}{\alpha}\right)s \\ p^* &= s, \quad \tilde{\theta}^* = \left(1 - \frac{s}{C}\right)^{1/\alpha} \text{ when } C > \left(1 + \frac{1}{\alpha}\right)s \end{aligned}$$

Fig. 1 plots the equilibrium price, market share, and profits of the proprietary software against C . To put the results in perspective, first suppose the open source software does not exist, and therefore the proprietary software vendor is a monopolist. The only constraint on the price of the proprietary software is that a user has to derive non-negative utility from the software. Hence, the vendor charges a monopoly price of $p^M = s$.

From Fig. 1, one can see that when open source software exists but the cost of deploying it is prohibitively high

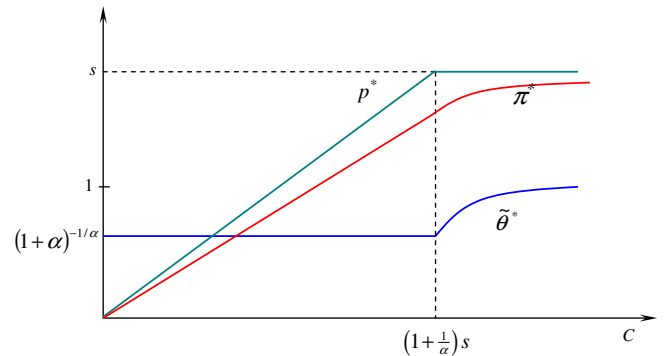


Fig. 1. Equilibrium price, market share, and profits for the proprietary software in the base case. Note: When C is below a threshold level, the two types of software have fixed market shares and the price of the proprietary software increases with C .

(i.e., $C > (1 + \frac{1}{\alpha})s$), the proprietary software vendor still charges the highest possible price s , because the open source software is not yet a serious threat to the proprietary software and it is profit maximizing for the vendor to keep the price as high as possible.⁹ As the cost for deploying open source software declines, the proprietary software gradually loses market share to the open source software. When the deploying cost for least skilled users falls below a threshold level (i.e., $C \leq (1 + \frac{1}{\alpha})s$), it becomes optimal for the vendor to lower the price of the proprietary software to prevent its market share from falling any further. As the cost for deploying S_2 keeps declining, the vendor is forced to keep cutting the price to maintain its market share, and in this process, it earns less profit.

The above result shows that the declining open source software deployment cost leads to a *shift in the proprietary software vendor's strategy*. When the open source software is costly to implement and support, it is optimal for the vendor to adopt a *high-price strategy*; when the maximum cost of deploying open source software drops below a threshold level, the vendor switch to the strategy of *maintaining market share by cutting price*.

2.3. Competition between systems with network effects

I now consider the case where a system, in addition to standalone value, also has network value, which is an increasing function of the mass of users adopting the same system. Since the total mass of users is normalized to 1, the mass of users adopting a system is the market share of that system. For a system with market share x , a user of this system derives a network value of $v(x)$. The functional form that is consistent with the Metcalfe's Law of network value is given by $v(x) = Bx^\gamma$, where the parameter B is the intensity of network effects [28].

More generally, network benefits tend to level off after a network reaches an adequate size. In other words, a network product usually has diminishing marginal network value to users. Therefore, a more general functional form of network value is:

$$v(x) = Bx^\gamma, \quad \gamma \in (0, 1]$$

Suppose the market share of S_1 is m . For any user of type θ , the net benefits from implementing S_1 and S_2 are $u_1 = s + v(m) - p$ and $u_2 = s + v(1 - m) - c(\theta)$. Again, the user indifferent between the two systems is denoted by $\tilde{\theta}$, now given by $p = c(\tilde{\theta}) + v(m) - v(1 - m)$. It can be easily proved that users with $\theta < \tilde{\theta}$ choose S_1 and others choose S_2 . Since the users are uniformly distributed between $[0, 1]$, the market share of S_1 is $\tilde{\theta}$, that is, $m = \tilde{\theta}$. So, $p = c(\tilde{\theta}) + v(\tilde{\theta}) - v(1 - \tilde{\theta})$.

This implies that the proprietary software can be priced higher than the standalone value of the software when it has more market share than the open source software,

because users gain higher network value from the software and are willing to pay for this extra value.

The vendor's profit maximization problem is now given by:

$$\begin{aligned} \max_p \quad & \pi = \tilde{\theta} \cdot p \\ \text{s.t.} \quad & 0 \leq \tilde{\theta} \leq 1, \quad 0 \leq p \leq s + v(\tilde{\theta}), \\ & \text{where } p = c(\tilde{\theta}) + v(\tilde{\theta}) - v(1 - \tilde{\theta}) \end{aligned}$$

The following closed-form solution can be derived when assuming that $c(\theta) = (1 - \theta^\alpha)C$ with $\alpha = 1$, and $v(x) = Bx^\gamma$ with $\gamma = 1$ (see also Fig. 2):

$$p^* = B, \quad \tilde{\theta}^* = 1 \text{ when } C \leq 3B$$

$$p^* = \frac{1}{2}(C - B),$$

$$\tilde{\theta}^* = \frac{1}{2} + \frac{B}{2(C - 2B)} \quad \text{when } 3B < C \leq 2B + s + \sqrt{B^2 + s^2}$$

$$p^* = \frac{1}{2}(B + s + \sqrt{B^2 + s^2}),$$

$$\tilde{\theta}^* = \frac{1}{2} + \frac{s}{2(s + \sqrt{B^2 + s^2})} \quad \text{when } C > 2B + s + \sqrt{B^2 + s^2}$$

To interpret these results, we must first look at the proprietary software vendor's pricing strategy. Similar to the base case, the vendor adopts a high-price strategy when the cost of deploying open source software is high, cuts the price when the cost is in the medium range, and keeps its price level at $p = B$ when the cost is low. Recall that in the base case, the price of the proprietary software drops from the monopoly price to zero as the maximum cost of deploying open source software declines. The presence of network effects minimizes the price change. On the one hand, the maximum price the proprietary software vendor charges in light of the competition from open source software is $p^* = \frac{1}{2}(B + s + \sqrt{B^2 + s^2})$, lower than $p^M = s + B$.¹⁰ On the other hand, the minimum price is now $p = B$, which means that the proprietary software vendor does not have to charge a price any lower than the network value (assuming it covers the entire market).

The market structure in the presence of network effects, however, is quite different from the base case. When the cost of deploying open source software is high, the proprietary software vendor charges a high price, *giving up* a large market share to the open source software. The vendor can afford to do so because the less skilled users are "stuck" with the proprietary software (the open source software is still too costly for them), and thus the vendor can make enough profit from these users. In other words, the vendor would rather lose consumers than reduce the price.

When the maximum cost of implementing the open source software falls below a certain level, however, the

⁹ Since $c(\theta) = (1 - \theta^\alpha)C$, the cost of deploying S_2 for any user is proportionate to C . Therefore, lower C also means lower cost for all users.

¹⁰ When the proprietary software vendor is a monopolist, the maximum price it can charge is given by $p \leq s + B\tilde{\theta}$ ($\tilde{\theta}$ denotes the market covered by the proprietary software). It can be easily proved that in this case, it is optimal for the vendor to cover the entire market and charge a monopoly price of $p^M = s + B$.

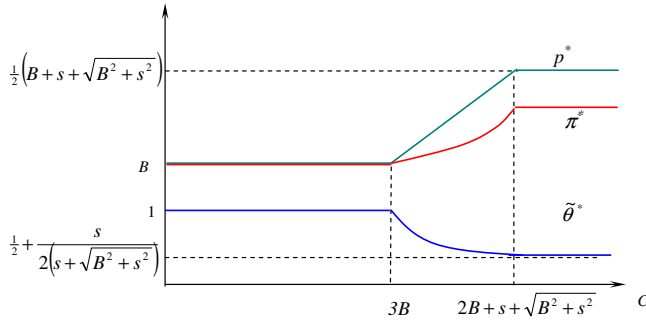


Fig. 2. Equilibrium price, market share and profits for the proprietary software in the presence of network effect ($\alpha = 1$, $\gamma = 1$). Note: When C is higher than $3B$, the two types of software share the market; when C is lower than $3B$, the proprietary software vendor takes the entire market.

proprietary software vendor changes its strategy: it tries to attract users and *gain market share* by reducing its price. The vendor keeps cutting the price as the cost for implementing the open source software declines, gaining market share until it drives the open source software out of market (see Fig. 2). Most surprising of all, the *proprietary software vendor takes the entire market when the cost of implementing open source software is low*.

Recall that in the base case where network effects are not considered, the proprietary software vendor cuts the price to maintain its market share, and the two types of software share the market. When the systems exhibit network effects, however, the proprietary software vendor is able to *gain* market share and eventually drive the open source software out of market by reducing the price. Network effects allow the vendor to charge merely for the network value of the software, leaving the standalone value as surplus to users.¹¹ In equilibrium, no user will choose the open source software: since the open source software provides no more standalone value than the proprietary software, each user prefers the latter except for the most skilled user ($\theta = 1$) who is indifferent.¹²

It must be noted, though, that the existence of the open source software lowers the profit the proprietary software vendor makes. Comparisons between the monopoly case and the case with competition from the open source soft-

ware show that in the latter case, the vendor's lost profit becomes users' surplus. Before the advent of the open source software, the price of the proprietary software and the vendor's profit are given by $p^M = s + B$ and $\pi^M = s + B$, and users have zero surplus; when the open source software is available, the price and profit of the proprietary software become $p^* = B$ and $\pi^* = B$ when it takes over the entire market, and users' surplus equals s .

The above results are derived for the parameter values of $\alpha = 1$ and $\gamma = 1$. For other parameter values, closed-form solutions are not available. Simulations show that as long as the network value is characterized by Metcalfe's Law ($\gamma = 1$), different values of α do not change the above results qualitatively (see Fig. 3a and b). In general, as the cost of implementing open source software declines, the proprietary software vendor lowers the price to gain market share until it takes over the entire market at a price that equals the network value. The parameter α only changes the threshold level of C under which the vendor can take the entire market. Specifically, the vendor will price the proprietary software to equal to the network value and cover the entire market when $C \leq \frac{3B}{\alpha}$. α also changes the rate at which the vendor reduces the price in response to the decline of the open source software implementation costs, that is, the slope of the curve for the equilibrium price p^* against C .

When the systems have diminishing marginal network value (i.e., $\gamma < 1$), simulation results show that for various values of α and γ , the vendor will always price the proprietary software such that it equals the network value, driving the open source software out of the market (see, e.g., Fig. 3c and d). This is because when $\gamma < 1$, the network value of a system increases rapidly when it is first introduced and has a small market share. If the vendor does not reduce the price of the proprietary software when the open source software is invented, allowing the open source software to take over some market share, it cannot regain the market by lowering the price later. Therefore, as soon as the open source software is available, the vendor will reduce the price until no users find the open source software more attractive than the proprietary software.

In summary, when the open source software is a perfect substitute for the proprietary software in its standalone value, network effects enable the vendor to price the proprietary software so low that it may drive the open source software out of market. It is then natural to ask whether the market structure will change when the open source software provides higher standalone value to users than the proprietary software.

2.4. Case discussions

The above analysis shows that in the presence of network effects, if the open source software is not perceived to offer higher benefits to its users, the proprietary software tends to dominate the market. The following mini-case illustrates these results.

¹¹ In Fig. 2, the proprietary software takes over the entire market when its price equals the network value, with the market as one network, that is, $p^* = B$ and $\tilde{\theta}^* = 1$. Users' utility is $u_1 = s + B\tilde{\theta}^* - p = s$.

¹² As long as users cannot coordinate, the proprietary software priced at B covering the entire market when $C \leq 3B$ is an equilibrium (i.e., $p^* = B$ and $\tilde{\theta}^* = 1$). If users can coordinate, there are three cases. First, when $2B < C \leq 3B$, this is still the equilibrium. Second, when $B \leq C \leq 2B$, the firm can temporarily offer the proprietary software for free to regain the entire market and then restore the equilibrium. Third, when $C < B$, if all users such that $\theta \geq \frac{B-C}{2B-C}$ can coordinate and switch to the open source simultaneously, then the proprietary software shares the market with the open source software; if even more users can simultaneously switch to OSS, then OSS will take over the entire market; if all users such that $\theta \geq \frac{B-C}{2B-C}$ fail to act simultaneously to switch to OSS, the proprietary system will still have the entire market. Since the coordination requirement is difficult to meet, the equilibrium can be considered stable.

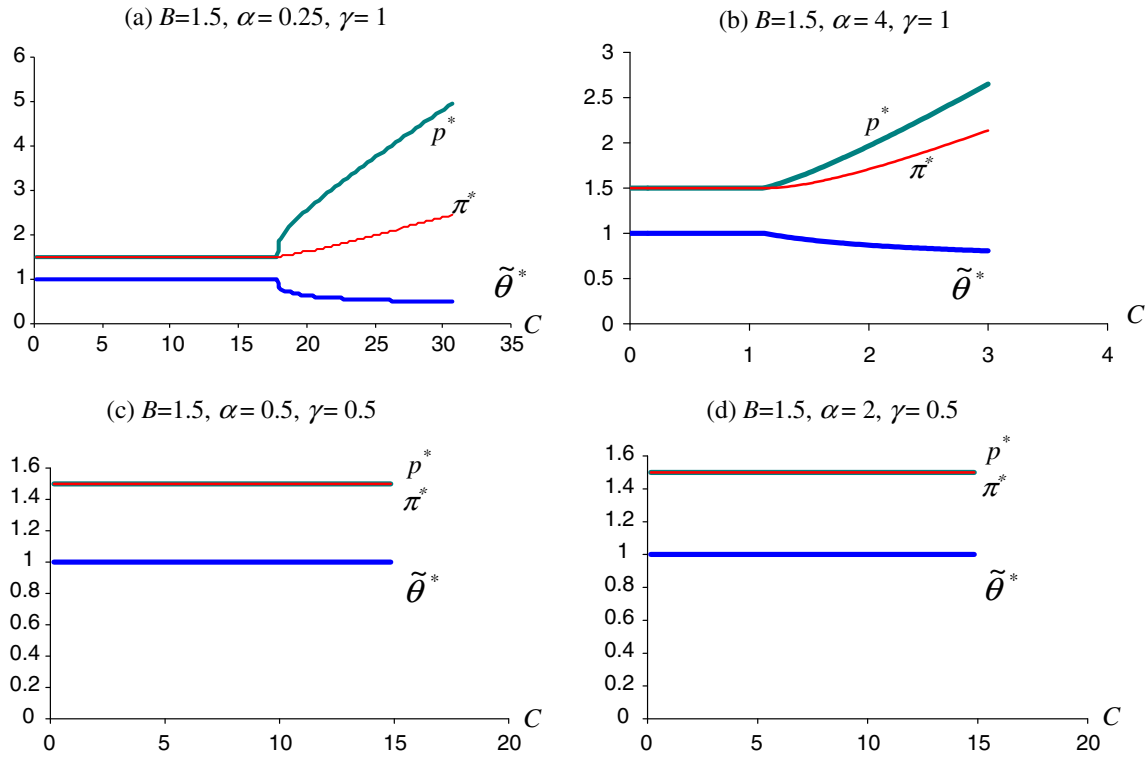


Fig. 3. Equilibrium price, market share and profit for the proprietary software in the presence of network effects. *Note:* When $\gamma = 1$, for different values of α , the above variables follow the same pattern as $\alpha = 1$. For $\gamma < 1$, however, for any value of α , the proprietary software vendor is the monopolist due to saturated network effects.

Mini-case 1: the web browser market in the 1990s. In the mid 1990s, the web browser market was characterized by competition between two proprietary products: Microsoft Internet Explorer (IE) and Netscape Navigator. Microsoft gained significant market share from Netscape in 1997. To counteract, in 1998 Netscape decided to stopping selling its Communicator software and to release the Communicator suite of applications under an open source license. This strategy, however, did not help Netscape regain its dominance. Results in this paper show that in presence of network effects, unless the open source software can provide significantly more benefits to users than its proprietary software counterpart, the proprietary software will dominate the market. While Microsoft's strategy to drive Netscape out of the market was challenged in court, one of the reasons for Netscape's failure was that it could not convince its potential users that its browser was superior to IE. Therefore, Microsoft could strategically set its price to attract users, taking advantage of the network value of software.

3. Competition between proprietary software and open source software with user customization of open source software

Users of open source software can customize the software to reap higher benefits. While proprietary software can also be customized to some extent, because its source code is closed, it offers much less flexibility than open

source software. This section studies the case where open source software can be customized while proprietary software cannot. However, as long as open source software is more customizable than proprietary software, the results hold (i.e., the level of proprietary software customization can be considered as normalized to zero).

3.1. The cost and benefits of customization for different types of users

A user can choose a certain level of open source software customization, $d \in [0, 1]$. Benefits from customization (such as performance improvement) are denoted by $r(d)$, which is a function of the level of customization. Therefore, while the standalone benefits from proprietary software remain the same as before, $s_1 = s$, the standalone benefits from open source software become $s_2 = s + r(d)$. Benefits from customization are often proportionate to the level of customization. Assuming a linear function, we have $r(d) = d \cdot R$, where R represents the highest benefits a user can derive from customization.

Customization incurs extra cost. The cost of deploying S_2 now becomes $c(\theta, d)$, which is a function of not only a user's type (level of skills) θ , but also the level of customization d . What properties should the cost function $c(\theta, d)$ have? First, it should satisfy $\frac{\partial c}{\partial \theta} < 0$, which means that for any given level of customization, users with more skills have lower costs. Second, the cost should increase with

the level of customization, which implies $\frac{\partial c}{\partial d} > 0$. Furthermore, it should satisfy $\frac{\partial^2 c}{\partial d \partial \theta} < 0$, which means that for a higher type user, the same increment in the level of customization incurs lower extra cost. I use a functional form that satisfies all the conditions above:

$$c(\theta, d) = (1 - \theta^\alpha)C + (1 - \theta^\alpha)d \cdot D$$

This specification suggests that the overall cost of deploying and customizing an open source system consists of two parts: the cost of deploying open source software to meet the basic requirements $(1 - \theta^\alpha)C$, and the cost of customization $(1 - \theta^\alpha)d \cdot D$. The user's skill level influences both parts of the costs: a user with high level of open source software skills incurs low costs both deploying basic functionalities and customizing the software.

In the cost function $c(\theta, d)$, the parameter C now stands for the cost of deploying the basic functionalities of the open source software for the least skilled user, $c(\theta = 0, d = 0) \equiv C$. The parameter D represents the *extra* cost of full customization incurred by the least skilled user, $c(\theta = 0, d = 1) \equiv C + D$. It should also be noted that $c(\theta)$ in Section 2 is in fact a special case of $c(\theta, d)$ where $d = 0$.

3.2. Competition between systems without network effects

A user's decision process occurs as follows: first, he decides which software (S_1 or S_2) to choose; next, if he chooses the open source software, S_2 , he then chooses the degree of customization. The optimal decisions are solved using backward induction. A user's optimal customization level (denoted by d^*) is determined by maximizing u_2 , then the software adoption problem is solved based on u_1 and the maximized u_2 .

To isolate the effect of customization, the case where S_1 and S_2 have no network value is discussed first. In the absence of network effects, the net utilities a type θ user gains from the two systems are given by $u_1 = s - p$ and $u_2 = s + r(d) - c(\theta, d)$.

We first solve for a type θ user's choice of customization level:

$$\max_d u_2 = s + r(d) - c(\theta, d)$$

Based on the assumption that $r(d) = d \cdot R$ and $c(\theta, d) = (1 - \theta^\alpha)(C + d \cdot D)$, the solution is given by:

When $R \geq D$, $d^* = 1$ for any $\theta \in [0, 1]$.

When $R < D$, $d^* = 1$ for $\theta > (1 - R/D)^{1/\alpha}$ and $d^* = 0$ for $\theta < (1 - R/D)^{1/\alpha}$.

We see that users either fully customize open source software or do not customize at all. $\hat{\theta}$ is defined such that $\hat{\theta} = (1 - R/D)^{1/\alpha}$ when $R < D$ and $\hat{\theta} = 0$ when $R \geq D$. Any user with $\theta < \hat{\theta}$ does not customize at all, while any user with $\theta > \hat{\theta}$ fully customizes.

Based on the customization levels they would choose, users adopt the software that maximizes their net utilities, that is, users choose the maximum of the following: $u_1 = s - p$ and $u_2 = s + r(d^*) - c(\theta, d^*)$. Again, we use $\hat{\theta}$ to denote the user who is indifferent between the two systems. We see that $\hat{\theta}$ is determined by $p = c(\hat{\theta}, d^*) - r(d^*)$. Considering the users' optimal adoption decisions, the proprietary software vendor chooses the profit-maximizing price. The equilibrium price and market share of the proprietary software are given in Table 2 (for more detailed analysis, see Appendix A).

In Section 2, the discussion focuses on how the market structure changes with users' costs of deploying open source software. When open source software customization is also considered, users evaluate the benefits and costs of adopting and customizing the open source software. As a result, the price and market share of the proprietary software depend on the relative magnitudes of the costs and benefits of open source software. Define the ratio $\rho \equiv R/D$, which represents the ratio between the benefits and costs of full customization for a user of type 0 (no open source software skills). Note that $\hat{\theta} = (1 - R/D)^{1/\alpha}$, which can be rewritten as $\hat{\theta} = (1 - \rho)^{1/\alpha}$. The higher ρ is, the lower $\hat{\theta}$, which means more users would choose to customize the open source software. Fig. 4 plots the equilibrium price, market share, and profit of the proprietary software against ρ , assuming C and D are fixed. (Therefore the change in ρ is due to the change in R only.) Results show that when $\rho \leq \frac{\alpha}{1+\alpha}$, the proprietary software and the open source software share the market (the price of the proprietary software and the market structure are the same as those in the base model). This means that when the benefits from customizing the open source software cannot justify

Table 2

Equilibrium price and market share of proprietary software in the case of no network effects and with open source software customization^a

Range of the ratio $\rho \equiv R/D$	Equilibrium price p^*	Equilibrium market share of proprietary software $\hat{\theta}^*$ and how $\hat{\theta}^*$ compares with $\hat{\theta}$
$\rho \leq \frac{\alpha}{1+\alpha}$	$\frac{\alpha}{1+\alpha}C$	$(1 + \alpha)^{-1/\alpha} < \hat{\theta}$
$\frac{\alpha}{1+\alpha} < \rho \leq \frac{\alpha}{\alpha + \frac{C}{C+D}}$	$\rho \cdot C$	$(1 - \rho)^{1/\alpha} = \hat{\theta}$
$\frac{\alpha}{\alpha + \frac{C}{C+D}} < \rho < 1 + \frac{C}{D}$	$\frac{\alpha}{1+\alpha}(C + D - R)$	$\left(\frac{1}{1+\alpha}\left(1 - \frac{R}{C+D}\right)\right)^{1/\alpha} > \hat{\theta}$
$\rho \geq 1 + C/D$	0	$0 = \hat{\theta}$

^a For succinctness, the upper limit for the price of the proprietary software is not shown. But it must be kept in mind that the price cannot exceed the standalone value, s .

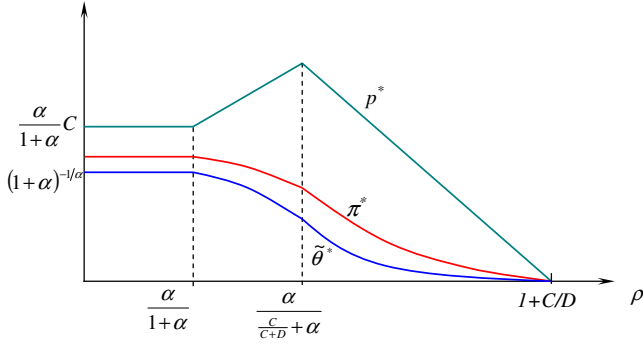


Fig. 4. Equilibrium price, market share, and profit for the proprietary software when the open source software is customizable ($C, D = \text{constant}$ and $\frac{\alpha C(C+D)}{C+\alpha(C+D)} \leq s$). †Note: The maximal price is reached when $\rho = \frac{\alpha}{\frac{C}{C+D} + \alpha}$ and $p_{\max} = \frac{\alpha C(C+D)}{C+\alpha(C+D)}$. Assume that the maximal price does not exceed the value of the system, s . When this assumption is violated, the price will be capped at s .

the costs for many users, the source code of open source software remaining open and modifiable does not increase adoption of the open source software. This is because only some, but not all, of the users who adopt the open source software will actually customize the system to enjoy the extra benefits ($\tilde{\theta}^* < \hat{\theta}$). Therefore, customizability is not a deciding factor in open source software taking market share away from proprietary software when customization is not justified for less skilled users.

As the ratio ρ increases, customization can be justified for more users. When the ratio is so high that all the users who have chosen S_2 will customize their system $\rho \geq \frac{\alpha}{1+\alpha}$, the proprietary software vendor finds it optimal to adopt the following strategy: it should give up market share so as to charge a high price. In fact, in the range $\frac{\alpha}{1+\alpha} < \rho \leq \frac{\alpha}{\frac{C}{C+D} + \alpha}$, the vendor charges a price such that the person indifferent between customization and no customization is also indifferent between the two systems ($\tilde{\theta}^* = \hat{\theta}$). That is, all users of the open source software customize their system, while no users of the proprietary software would customize the open source software had they chosen open source software. By focusing on this part of the market, which shrinks as ρ increases, the vendor can actually raise the price for the proprietary software (see Fig. 4).

When ρ rises to a higher level ($\rho > \frac{\alpha}{\frac{C}{C+D} + \alpha}$), so many users find it worthwhile to customize the open source software that the vendor of the proprietary software cannot afford to focus only on those who would not customize the open source software. Therefore, the vendor changes its strategy again, lowering its price to attract users who would actually customize the open source software if they chose it. However, the vendor continues to lose market share despite its lower prices. When the maximum benefits from customization exceed the maximum total cost of implementing and customizing the open source software, the proprietary software will be driven out of market.

3.3. Competition between systems with network effects

We now discuss the case where both systems have network effects and S_2 can be customized. When the market share of S_1 is $\tilde{\theta}$, for a type θ user, the utilities for choosing S_1 and choosing S_2 with a customization level of d are given by:

$$u_1 = s + v(\tilde{\theta}) - p$$

$$u_2 = s + v(1 - \tilde{\theta}) + r(d) - c(\theta, d)$$

By backward induction, we first solve for a user's optimal customization level for S_2 :

$$\max_d u_2 = s + v(1 - \tilde{\theta}) + r(d) - c(\theta, d)$$

Since each user maximizes its utility taking $\tilde{\theta}$ as given, users' customization decisions are the same as in Section 3.2, and the definition of $\tilde{\theta}$ remains the same.

Next, we solve for the users' adoption decisions and then the proprietary software vendor's pricing choice, taking into account users' decisions (see Appendix B for the formulation).

In general, results show that when the benefits of customizing the open source software exceed the costs by a large margin, open source software may take market share away from the proprietary software vendor and even drive the proprietary software out of the market. Results in Section 2 show that the proprietary software vendor can take advantage of the network effect by strategically pricing its product. To overcome this, the net customization benefits from open source software must be much higher, so as to attract users to adopt and customize open source software.

How the market structure changes with the benefits and costs of customization depends on the intensity of network effects. When the intensity of network effects is low (i.e., $B \leq [C + D]/3$), the strategic advantage of the proprietary software vendor is less significant; therefore as more users find it worthwhile to customize the open source software, the market for the proprietary software shrinks until it is driven out of market (when $\rho \geq \max\{1, 1 + \frac{C-B}{D}\}$).

It is more interesting to investigate the case where the network intensity is moderate or high. Fig. 5 depicts the equilibrium price and market share of the proprietary software against the ratio ρ when the intensity of network effects is in the medium range (i.e., $(C + D)/3 < B < (C + D)/2$ and $B > C$). We see that when ρ is positive but low, $\tilde{\theta}^* = 1$. This means that even when the benefits exceed the costs of customization for a positive mass of skilled users, the proprietary software still takes the *entire market*. In other words, the market structure is the same as in the case where open source software customization is not considered (Section 2.3). However, the proprietary software vendor cannot keep the same price as in the case without customization: in Section 2.3, the vendor sets $p^* = B$ to be a monopolist; with customization, however, as more users find customization beneficial, the vendor

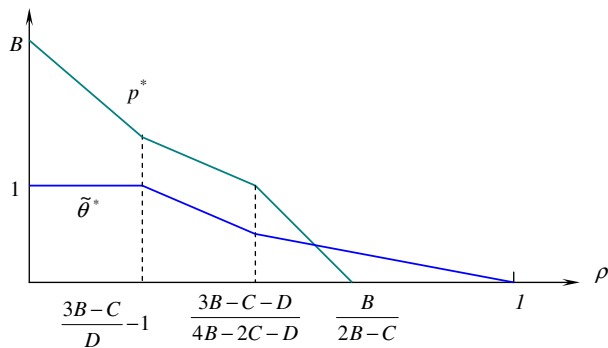


Fig. 5. Equilibrium price and market share for the proprietary software in the presence of network effects when open source software customization is considered ($B > C$, $(C + D)/3 < B < (C + D)/2$, and $s \geq B$). Note: The proprietary software vendor is the monopolist for low levels of ρ . The two types of software share the market only when ρ is above a threshold level. Higher ρ means more users find it worthwhile customizing the open source software.

has to reduce the price to remain the monopolist (see Fig. 5).

As the benefits of customization can justify the cost for more and more users, the proprietary software vendor continues to reduce the price, but starts to lose part of its market. When $\rho < \frac{3B-C-D}{4B-2C-D}$, the proprietary software is still able to keep some, but not all, of the users who are skilled enough to find customizing open source software beneficial. When $\rho > \frac{3B-C-D}{4B-2C-D}$, however, the proprietary software has to appeal to users who are less skilled and would choose not to customize the open source software. As the ratio ρ rises, this part of the market shrinks, and the vendor also has decreasing market power to charge a high price. Eventually, the market tips towards the open source software.

Results that are even more dramatic occur when the intensity of network effects is high. When $B > (C + D)/2$ and $B > D$, the proprietary software will take the entire market as long as the full benefits of customizing the open source software cannot compensate for the network effects from the proprietary software (i.e., $R < B$). To keep the market for itself, however, the proprietary software vendor has to reduce the price as R increases. The price will be reduced to zero when $R = B$, which means that the vendor is making zero profit, regardless of its market share. With higher benefits of customizing the open source software, the market suddenly tips towards the open source software.

3.4. Case discussions

Our theoretical results highlight the importance of the heterogeneity of users' open source software skills for competition between open source and proprietary software. In the presence of network effects, the open source software may penetrate a proprietary software-dominated market only if a significant portion of users finds customization worthwhile. In such a market, the proprietary software may lose market share despite a reduced price.

Next, we discuss several mini-cases to see how the above results help us better understand the competition between open source and proprietary software.

3.4.1. Mini-case 2: recent developments in the web browser market

The recent success of the open source web browser Mozilla Firefox is an example of open source software winning market share because of its benefits to skilled users. It is widely believed that Mozilla Firefox is more secure than Microsoft IE, and it also allows users to customize the browser, which is valued by skilled users. Market reports have shown that the market share of Firefox has been steadily increasing since its release in 2004 [8].

3.4.2. Mini-case 3: database management systems market

Several surveys show that open source database management systems (DBMS) such as MySQL have gained significant market share recently [29]. Some industry analysts believe that databases, rather than operating systems, are now driving the adoption of open source in enterprises. One of the main reasons is that open source DBMS are improving in performance and scalability. Since users of enterprise DBMS tend to have the skills to deploy and customize open source DBMS, it is not surprising that this system has gained market share over competing proprietary software.

3.4.3. Mini-case 4: web server market

Perhaps the most successful open source software, in terms of market share, is the Apache web server software. Netcraft's surveys on web servers have shown Apache dominating the public Internet web server market since Apache reached the No.1 position in April 1996 [34]. In a November 2005 survey, Apache had 71.0% of the market while Microsoft IIS had 20.2%. However, in the most recent survey, dated November 2006, Apache's market share plummeted to 60.3%, while Microsoft's share gained substantially, reaching 31.0% [34].

Such results cannot be explained with models that abstract from the composition of users and their skills levels. Until 2005, typical users of web server software were "webmasters", individuals or groups responsible for the operation of organizational websites. These highly skilled users tended to choose Apache because they could modify the source code to meet their specific needs [17]. In the last two years, however, the number of websites has increased drastically, with blogs and small business sites driving explosive growth, according to Netcraft. These users tend to be less skilled than typical webmasters.

Our results predict that when less-skilled users enter the market, given the strategic pricing of the proprietary software vendor, the market share of the proprietary software may rise. The changing market structure of the web server market highlights the importance of the heterogeneity of users' skills in the competition between proprietary software and open source software.

3.4.4. Mini-case 5: operating systems market: Linux vs. Windows

The competition between Linux and Windows in the operating systems market has attracted much attention. One challenge to conducting research on this market is the lack of unbiased empirical data: surveys of market share are often sponsored either by proprietary software vendors or by the open source community.

Linux has been more successful in the server operating systems (OS) segment than in the desktop segment. This is hardly surprising, considering that users of server OS are often IT professionals who are more likely to have open source software skills. The desktop OS market is still dominated by Microsoft Windows, with Linux keeping a market share of approximately 3.3% in 2005 and 2006 [44]. Our analysis predicts that without drastic changes in the open source software skills of the user population, the market structure of the desktop OS market will remain largely stagnant in the coming years, assuming no major breakthroughs or breakdowns in either OS.

4. Concluding remarks and future research

Given the assumption of heterogeneous users, this paper studies the impact of network effects and user customization on the market structure when proprietary software competes with open source software. We show that the market may tip towards open source software only if it provides significant benefits to users who can customize the open source software. It implies that for open source software to survive in an industry characterized by network effects, mere cost advantage is not enough. System customization, flexibility, and better performance are the key to success.

This result is confirmed by a 2005 survey of European end-users conducted by IDC [5]. Companies surveyed did not cite low cost as their main reason for deploying open source, a factor usually considered one of the main reasons for open source's success. Rather, companies said open source's top benefit was its flexibility. Many companies also said the ability to customize open-source software was important. Interestingly, IDC did not suggest "customization" as one of the standard multiple-choice answers; instead, many companies added it in the "comments" section of the survey.

IT professionals skilled in open source software are the driving force behind the adoption and success of open source, because they are able to develop tools for users, making open source systems more valuable to them. The Sakai project is another example that exemplifies this. The Sakai project, using an experimental variation of an open source development model called the community source model, has been growing quickly since its start in 2004. Universities are switching from Blackboard to Sakai because they value the customizability of Sakai and have the skills to take advantage of it.

The results of this paper suggest that open source software providers should not restrict their attention to lowering the implementation and maintenance costs of open source software. Instead, they should focus on providing value-adding services to customers. This implies that commercially providing value-adding services and toolkits for users of open source software will be a successful strategy for companies trying to capture value from open source software. In fact, this has already been recognized as one of the business models that can be adopted by open source software providers. Raymond [35] calls it the "give-away-the-recipe, open-a-restaurant" model and it is also mentioned by Lerner and Tirole [25]. The success of companies such as Red Hat shows the viability of such a strategy. This paper provides a theoretical foundation for this strategy and suggests further implications. Firms providing support and services for open source software usually have to release the source code for any modifications and improvements, making their innovations a commodity [10,31]. However, the knowledge and skills these open source software providers gain by implementing and customizing systems for clients are often difficult to replicate and therefore can serve as their competitive advantage.

In theory, users can customize open source software by modifying its source code. In practice, however, users who choose open source software offered by an open source software provider often rely on the provider for customization. Our results imply that making customization less costly for users will increase the market share of open source software. Therefore, it may be in the open source software providers' interest to offer tools for open source software customization to users.

Our results also suggest that as open source software becomes more cost efficient to be implemented and customized, the price and profit of proprietary software tends to decline. The pricing of software is very complex, due to software's multiple features, various licensing terms, etc. Careful empirical research is needed to better understand the impact of competition from open source software on the price and profit of proprietary software.

This research considers the impact of user skills, network effects, and open source software customization on the market structure. Nevertheless, there are other important factors influencing these markets. For example, a 2005 survey by Open Source Development Labs (OSDL) finds that the lack of support for applications and peripherals are the top "inhibitors" of Linux adoption. While this issue is related to the network effects of software, further research about the interactions between application vendors and open source software platforms will be valuable for understanding not only open source software but also open standards in general.

Acknowledgements

The author thanks Rob Kauffman, co-editor in chief of *Electronic Commerce Research and Applications*, Zhangxi

Lin and Karl Lang, co-editors of the special issue, and four anonymous referees. The author also appreciates Bin Gu, Bala Iyer, Nalin Kulatilaka, Chi-Hyon Lee, Michael Salinger, Yu Wang, participants of HICSS'39 conference and seminars at Boston University, University of Illinois at Urbana-Champaign, and University of Texas at Austin for insightful comments. Research assistance provided by Michelle Xiang Liu and Nicolas Beaulieu is grateful acknowledged. Partial support was provided by the Junior Faculty Research Fund at Boston University. All errors and omissions are the author's.

Appendix A. Model formulation when users can customize the open source software: network effects not considered

First, consider the case $R \geq D$, where all users would fully customize the open source software if they choose it (i.e., $\hat{\theta} = 0$). Since $d^* = 1$ for all users, $\tilde{\theta}$ is given by $p = (1 - \tilde{\theta}^x)(C + D) - R$.

Next, consider the case of $R < D$, where $d^* = 1$ for $\theta > \hat{\theta}$ and $d^* = 0$ for $\theta < \hat{\theta}$. If the user indifferent between the two types is one who fully customizes S_2 (i.e., $\hat{\theta} > \tilde{\theta}$), its type is again determined by $p = (1 - \tilde{\theta}^x)(C + D) - R$. If the indifferent user does not customize (i.e., $\hat{\theta} < \tilde{\theta}$), its type will be determined implicitly by $p = r \cdot 0 + c(\tilde{\theta}, 0) = (1 - \tilde{\theta}^x)C$.

Based on the above analysis, we now turn to the decision of the proprietary software vendor. When $\hat{\theta} > \tilde{\theta}$, the vendor's profit maximizing problem is given by:

$$\begin{aligned} \max_p \quad & \pi = \tilde{\theta} \cdot p \\ \text{s.t.} \quad & 0 \leq \tilde{\theta} \leq 1 \text{ and } 0 \leq p \leq s, \\ & \text{where } p = (1 - \tilde{\theta}^x)(C + D) - R \end{aligned}$$

When $\hat{\theta} < \tilde{\theta}$, the vendor's profit maximizing problem is given by:

$$\begin{aligned} \max_p \quad & \pi = \tilde{\theta} \cdot p \\ \text{s.t.} \quad & 0 \leq \tilde{\theta} \leq 1 \text{ and } 0 \leq p \leq s, \quad \text{where } p = (1 - \tilde{\theta}^x)C \end{aligned}$$

Solving the maximization problems, considering different ranges of parameters, yields the results in Table 2.

Appendix B. Model formulation when users can customize the open source software: both proprietary and open source software exhibit network effects

Following previous cases, the functional forms used are: $v(x) = Bx^\gamma$, $r(d) = d \cdot R$ and $c(\theta, d) = (1 - \theta^x)(C + d \cdot D)$. Results have shown that different values for the parameters α and γ do not change the features of the results. To gain insights without loss of generality, results are derived assuming $\alpha = 1$ and $\gamma = 1$. When $\hat{\theta} > \tilde{\theta}$, which means that the user indifferent between the two systems customizes S_2 , the price of S_1 is determined implicitly by:

$$p = C + D - B - R + (2B - C - D)\tilde{\theta}$$

The vendor's profit maximizing problem is:

$$\begin{aligned} \max_p \quad & \pi = \tilde{\theta} \cdot p \\ \text{s.t.} \quad & 0 \leq \tilde{\theta} \leq 1, 0 \leq p \leq s + B\tilde{\theta}, \\ & \text{where } p = C + D - B - R + (2B - C - D)\tilde{\theta} \end{aligned}$$

When $\hat{\theta} < \tilde{\theta}$, the indifferent user does not customize S_2 , thus the vendor's problem is in effect the same as that in Section 2.3.

These maximization problems are solved under different cases of parameter values. The complete set of results is available upon request.

References

- [1] J. Bessen, Open source software: free provision of complex public goods, Unpublished working paper, Research on Innovation, 2004. Available from: <<http://researchoninnovation.org>>.
- [2] A. Bonaccorsi, C. Rossi, Why open source software can succeed? Research Policy 32 (7) (2003) 1243–1258.
- [3] G.H. Bower, E.R. Hilgard, Theories of Learning, Prentice-Hall, Englewood Cliffs, NJ, 1981.
- [4] T.F. Bresnahan, Network effects and Microsoft, Stanford Institute for Economic Policy Research Discussion Paper No. 00-51, 2001.
- [5] M. Broersma, IDC: Quality drives European open-source adoption, Techworld.com, 2005. Available from: <<http://www.linuxworld.com.au/index.php/id/1623210212;fp;16;fpid;0>>.
- [6] E. Brynjolfsson, C. Kemerer, Network externalities in microcomputer software: an econometric analysis of the spreadsheet market, Management Science 42 (12) (1996) 1627–1647.
- [7] R. Casadesus-Masanell, P. Ghemawat, Dynamic mixed duopoly: a model motivated by Linux vs. Windows, Management Science 52 (7) (2006) 1072–1084.
- [8] T. Claburn, Firefox eats more Microsoft share, InformationWeek, March 18, 2005. Available from: <<http://www.informationweek.com/story/showArticle.jhtml?articleID=159902316>>.
- [9] W.M. Cohen, D.A. Levinthal, Absorptive capacity: a new perspective on learning and innovation, Administrative Science Quarterly 35 (1) (1990) 128–152.
- [10] M.A. Cusumano, Reflections on free and open software, Communications of the ACM 47 (10) (2004) 25–27.
- [11] Cybersource, Linux vs. Windows total cost of ownership comparison, 2004. Available from: <http://www.cyber.com.au/about/linux_vs_windows_tco_comparison.pdf>.
- [12] J.S. David, D. Schuff, R.S. Louis, Managing your IT total cost of ownership, Communications of the ACM 45 (1) (2002) 101–106.
- [13] N. Economides, E. Katsamakos, Two-sided competition of proprietary vs. open source technology platforms and the implications for the software industry, Management Science 52 (7) (2006) 1057–1071.
- [14] J. Feller, B. Fitzgerald, A framework analysis of the open source software development paradigm, in: W. Orlikowski, P. Weill, S. Ang, H. Krcmar (Eds.), Proceedings of the 21st Annual International Conference on Information Systems, 2000, pp. 58–69.
- [15] J. Feller, B. Fitzgerald, Understanding Open Source Software Development, Addison-Wesley, London, 2002.
- [16] J. Feller, B. Fitzgerald, S.A. Hissam, K.R. Lakhani (Eds.), Perspectives on Free and Open Source Software, MIT Press, Cambridge, MA, 2005.
- [17] N. Franke, E. von Hippel, Satisfying heterogeneous user needs via innovation toolkits: the case of apache security software, Research Policy 32 (7) (2003) 1199–1215.
- [18] A. Gaudeul, Competition between open-source and proprietary software: the LaTeX case study, Unpublished working paper, Universities of Toulouse and Southampton, 2004.

- [19] J. Giera, The costs and risks of open source, Forrester Research, 2004. Available from: <http://www.forrester.com/Research/Document/Excerpt/0,7211,34146,00.html>.
- [20] L. Greenemeier, J. Foley, In search of experts, *InformationWeek*, November 18, 2004, pp. 26–42.
- [21] Info-Tech Research Group, 2005 IT priorities, 2005. Available from: <http://www.infotech.com>.
- [22] J.P. Johnson, Open source software: private provision of a public good, *Journal of Economics and Management Strategy* 11 (4) (2002) 637–662.
- [23] M.L. Katz, C. Shapiro, Technology adoption in the presence of network externalities, *Journal of Political Economy* 94 (4) (1986) 822–841.
- [24] D. Kusnetzky, A. Gillen, Linux: a journey into the enterprise, IDC whitepaper, 2001.
- [25] J. Lerner, J. Tirole, Some simple economics of open source, *Journal of Industrial Economics* 50 (2) (2002) 197–234.
- [26] J. Lerner, J. Tirole, The economics of technology sharing open source and beyond, *Journal of Economic Perspectives* 19 (2) (2005) 99–120.
- [27] S.J. Liebowitz, S.E. Margolis, Network externality: an uncommon tragedy, *Journal of Economic Perspectives* 8 (2) (1994) 133–150.
- [28] L. Lin, N. Kulatilaka, Network effects and technology licensing with fixed fee, royalty, and hybrid contracts, *Journal of Management Information Systems* 23 (2) (2006) 91–118.
- [29] V. McCarthy, Evans: IT using more open source databases, 2005. Available from: http://oetrends.com/news.php?action=view_record&idnum=392.
- [30] Meta Group, Linux servers: no silver bullet for total cost of ownership, 2002. Available from: http://download.microsoft.com/download/8/6/d/86da6dab-b245-46db-b4e8-f2183c1b6584/meta_tco.pdf.
- [31] V. Murphy, Open season, *Forbes*, April 19, 2004.
- [32] M. Mustonen, Copyleft – the economics of Linux and other open source software, *Information Economics Policy* 15 (1) (2003) 99–121.
- [33] Netcraft, September 2001 web server survey. Available from: <http://survey.netcraft.com/Survey/index-200109.html>.
- [34] Netcraft, November 2006 web server survey. Available from: http://news.netcraft.com/archives/web_server_survey.html.
- [35] E.S. Raymond, *The Cathedral and the Bazaar: Musing on Linux and Open Source by an Accidental Revolutionary*, O'Reilly, Sebastopol, CA, 2001.
- [36] Robert Frances Group, Total cost of ownership for Linux in the enterprise, 2002. Available from: <http://www-1.ibm.com/linux/RFG-LinuxTCO-vFINAL-Jul2002.pdf>.
- [37] S. Rosen, Learning by experience as joint production, *Quarterly Journal of Economics* 86 (3) (1972) 366–382.
- [38] K. Schmidt, M. Schnitzer, Public subsidies for open source? Some economic policy issues of the software market. Discussion Paper No. 3793, Center for Economic Policy Research, 2003.
- [39] R. Schmidtke, Private provision of a complementary public good, CESifo Working Paper Series No. 1756, July 2006. Available from: <http://ssrn.com/abstract=920342>.
- [40] C. Shapiro, H. Varian, *Information Rules: A Strategic Guide to the Network Economy*, Harvard Business School Press, Boston, MA, 1999.
- [41] M. Shurmer, An investigation into sources of network externalities in the packaged PC software market, *Information Economics and Policy* 5 (3) (1993) 231–251.
- [42] Soreon Research, Saving cash: a comparison of open source and proprietary software, 2004. Available from: <http://www.research-handmarkets.com/reports/c8216/>.
- [43] G. von Krogh, E. von Hippel, The promise of research on open source software, *Management Science* 52 (7) (2006) 975–983.
- [44] W3Schools, Web statistics and trends, 2006. Available from: http://www.w3schools.com/browsers/browsers_stats.asp.
- [45] S.A. Zahra, G. George, Absorptive capacity: a review, reconceptualization, and extension, *Academy of Management Review* 27 (2) (2002) 185–203.